多智能体驱动的嵌入式代码检测系统

实　施　方　案

项　目　名　称：**嵌入式代码检测系统**

编　制　单　位：**嵌入式代码检测系统项目组**

编　制　依　据：**《技术调研报告》V1.0（2026-09）**

文　档　版　本：**V1.1（简洁版）**

完　成　日　期：**2026 年 9 月**

目　录

第 1 章　测什么：查哪些问题 2

1.1 检测清单怎么定 2

1.2 先查哪类缺陷 3

1.3 用哪些规范当尺子 3

1.4 安全标准的影响 3

1.5 四个维度怎么做：一个一个来 4

第 2 章　怎么分：四类方法与第一期工具 4

2.1 四类方法各管什么 4

2.2 第一期用什么工具 4

2.3 告警统一说一种话：SARIF 5

2.4 动态检测为什么放后面 5

第 3 章　怎么干：架构与计划 5

3.1 总路线：核心自己写，其余全复用 5

3.2 系统分三层 6

3.3 四个阶段 6

3.4 第一个月（MVP）四周计划 7

3.5 能复用的开源项目 7

3.6 模型怎么选、钱怎么省 7

3.7 效果怎么评.....8

3.8 风险与对策.....8

第 4 章　本周就能开始的事.....8

4.1 三件事，不用写代码.....8

4.2 需要拍板的三个问题.....9

注：本目录由域代码生成。编辑文档后，请在目录上单击右键选择“更新域”刷新页码。

第 1 章　测什么：查哪些问题

本方案回答三个问题：查什么、怎么查、怎么干。核心结论先说：第一期只做静态缺陷检测——用现成开源工具把问题查出来，再用大模型把误报挑掉；规范检查、测试生成等内容后面分阶段上。

1.1 检测清单怎么定

清单按两个维度整理。一是缺陷类型：按国际通用的 CWE 编号归类（越界写、空指针、内存泄漏等）；二是规范条款：MISRA C、GJB 8114、CERT C 里一条条具体的编程规则。两个维度对得上号——每条规则背后都是一类缺陷。最终每条检测项就是四件事：查什么缺陷、违反哪条规则、用哪个工具查、有多严重。行业标准（比如汽车的 ISO 26262）的作用是裁剪：安全等级越高，要查的项越多。

1.2 先查哪类缺陷

看真实漏洞数据说话：2025 年 CWE Top 25 的已知被利用（KEV）子榜里，越界写得分 50.39，约为第二名 XSS（17.99）的三倍。所以内存安全排第一。表 1-1 列出七类缺陷和嵌入式场景下的典型成因。

表 1-1 七类缺陷与嵌入式典型成因（按优先级排列）

缺陷类别	常见问题	嵌入式里为什么高发
内存安全	缓冲区溢出、数组越界、空指针、野指针、内存泄漏	解析协议数据时无界拷贝；没有内存保护的芯片越界写不报错、悄悄破坏别的任务；设备常年不关机，小泄漏也会攒成大事故
并发与中断	数据竞争、死锁、优先级反转、中断函数不可重入	中断和任务共用变量却没加保护；volatile 不保证原子性，多字节数据可能读一半被改
整型与算术	有符号溢出、隐式截断、除零	16 位单片机上 int 只有 16 位，类型转换时容易悄悄截断
未定义行为	严格别名、对齐错误、悬空指针	不同编译器对同一段有歧义的代码处理不同，换个编译器就出故障
栈与实时性	栈溢出、函数执行超时	递归、大局部数组、中断嵌套把栈吃穿

缺陷类别	常见问题	嵌入式里为什么高发
字符串与资源	不安全字符串函数、外设句柄泄漏	错误处理分支忘了关外设；安全标准普遍禁止运行时动态分配内存
结构与质量	死代码、圈复杂度超标	影响认证追溯和长期维护

1.3 用哪些规范当尺子

规范就是一把把尺子，要按目标标准分库维护。两个现实问题要注意：最新的 MISRA C:2025 开源工具还查不了（cppcheck 的开源插件只支持 2012 版，2025 版只有商业工具支持）；GJB 8114 和 GJB 5369 两套国军标规则编号互不通用，规则库得分开建。

表 1-2 规范基线现状

规范	内容	开源工具支持情况
MISRA C:2012（含修订）	约 200 条规则，汽车电子行业主流	cppcheck、clang-tidy 可查
MISRA C:2025	约 225 条规则（2025 年新版）	开源暂不支持，仅商业工具
SEI CERT C	98 条安全编码规则，与 CWE 有映射	部分支持
GJB 8114 / GJB 5369	国内军品事实基线（124 条 / 138 条）	国产工具或自建规则库

1.4 安全标准的影响

标准的影响就三条：安全等级越高，静态分析越强制；覆盖率要求越高，最高到 MC/DC（修正条件判定覆盖，航空最高等级强制）；静态查完还得动态测，两者不能互相替代。落到系统里：标准决定检测项开多少，用配置文件控制。

1.5 四个维度怎么做：一个一个来

四个维度不齐头并进，按把握从大到小、难度从低到高排序：静态缺陷第一（工具成熟，去误报正是大模型擅长的事）；规范合规第二（建规则库费人力但结果确定）；测试生成第三；并发时序最后（工具最难、误报最多）。固件检测放二期。这个顺序在第 3 章的计划里落实。

第 2 章　怎么分：四类方法与第一期工具

2.1 四类方法各管什么

检测方法分四类，各有长短：静态分析不运行程序、直接读代码找问题，查得全但会误报；动态分析把程序真跑起来观察，准但查不到没执行到的路径；混合分析自动生成测试用例、能钻到深层路径，但算力开销大；固件分析对付没有源码的二进制。第一期只用静态分析，其余按阶段引入。

表 2-1 四类方法与引入阶段

方法	长处	短板	什么时候用
静态分析	查得全，能查规范违规	有误报，需要人工或大模型分辨	第一期（本方案主线）
动态分析	查得准，能拿到覆盖率证据	查不到没跑到的代码；裸机限制多	第三阶段
混合分析	自动造测试用例，钻深路径	算力贵、路径爆炸	后备，按需用 CBMC/KLEE
固件分析	查无源码组件的漏洞	定制芯片难仿真	二期扩展

2.2 第一期用什么工具

选工具只有两条硬标准：能命令行调用；结果能输出成 JSON/XML 这类机器可读格式——大模型系统要把它们当工具使唤。第一期选型见表 2-2，全部开源、都在活跃维护。

表 2-2 第一期工具链

工具	干什么用	结果格式	许可证
cppcheck 2.22	规则检查，带 MISRA 插件	XML / SARIF	GPL-3.0
clang-tidy	代码规则检查与自动修复	YAML（可转 JSON）	Apache-2.0
GCC -fanalyzer	路径分析，交叉编译工具链自带	JSON / SARIF	GPL-3.0
CBMC（后备）	数学级验证高危模块	原生 JSON	BSD 类
Ceedling/Unity/CMock（后续）	单元测试与硬件打桩	JUnit XML	MIT
Renode（后续）	单片机仿真	XML	MIT

2.3 告警统一说一种话：SARIF

三个工具输出的格式各不相同，统一转成 SARIF——静态分析结果的通用交换标准，相当于这个领域的普通话。统一之后，大模型研判层只认一种格式，以后加新工具也不用改研判层，这是整个系统保持简单的关键。

2.4 动态检测为什么放后面

有个绕不开的限制：ASan、Valgrind 这类内存检测工具依赖操作系统，裸机单片机上用不了（只有 UBSan 可以裁剪后用）。所以裸机的动态测试要走替代路线：在电脑上用 CMock 把硬件相关代码换成替身，跑 Unity 单元测试；需要模拟外设行为时用 Renode 仿真器。这些放到第三阶段做，本期不投入。

第 3 章　怎么干：架构与计划

3.1 总路线：核心自己写，其余全复用

总路线一句话：核心自己写（量不大），其余全部用现成开源，不 fork 大项目当底座三个理由：第一，系统的价值在懂嵌入式的部分——工具适配、规则库、合规报告，这些现成项目里都没有，只能自己写；而这部分写完后，大模型编排只是薄薄一层。第二，最像的候选 open-code-review（4.3 万 star）是 Go 语言写的 PR 审查工具，fork 它等于接下整个 Go 技术栈，还换不来核心能力。第三，自研总量约几千行 Python，完全可控。open-code-review 的正确用法是当对照组：跑同一份代码，我们必须证明比通用方案更懂嵌入式。

3.2 系统分三层

系统分三层，见图 3-1：工具层把问题查准查全；智能体层读懂告警——并行分诊、每条结论附代码证据、高危项留给人工确认；报告层汇总交付。哪些自研、哪些复用见表 3-1。



图 3-1 系统总体架构（深色为自研，浅色为复用）

表 3-1 组件清单：自研还是复用

组件	来源	说明
工具适配器（三个静态工具）	自研（薄）	每个约百余行：调用工具、解析结果、转成 SARIF
SARIF 告警池	自研	统一存放告警，附带函数和调用链上下文
编排与分诊（LangGraph）	自研（核心）	并行分诊、确定汇总、高危项转人工
PR 评论回写	复用 reviewdog（MIT）	把告警变成代码平台上的行内评论，省掉接口开发
检测与合规报告	自研	按 MISRA/GJB/ISO 26262 条款生成
模型底座	外部服务	国产大模型 API；内网部署时换 vLLM 本地跑
效果对照	复用 open-code-review	只用来对比效果，不引入其代码

3.3 四个阶段

表 3-2 分阶段路线

阶段	做什么	做完的标志
一（本期 MVP）	静态缺陷检测与告警分诊跑通	去误报率等指标有实测数据

阶段	做什么	做完的标志
二	规范合规检查（MISRA/GJB）	对目标工程出完整合规报告
三	单元测试生成与覆盖率	覆盖率提升可量化
四	并发时序检测与多项目部署	嵌入式特色能力可用

3.4 第一个月（MVP）四周计划

MVP 只干一件事：对目标工程的静态告警做端到端分诊，拿出实测数据。目标工程建议用 FreeRTOS 或 RT-Thread 的官方示例（真实、规模适中、有已知缺陷历史）。

表 3-3 四周计划与验收标准

周	干什么	验收标准
第 1 周	搭项目骨架；选定目标工程；写三个工具适配器，告警统一进 SARIF 池	一条命令完成全量扫描，告警带文件、行号、CWE 编号
第 2 周	搭 LangGraph 流水线和分诊智能体：读告警上下文，判误报、定级、给原因和修复建议，附代码证据	抽查 50 条，每条都能对上代码证据
第 3 周	出检测报告第一版；接 reviewdog 发 PR 评论；跑 open-code-review 做对照	一条命令出完整报告，与对照有明确差异
第 4 周	接入 Juliet 评测集，统计指标、写回归脚本、算成本	去误报率、漏判率、单条成本三项指标可复现

3.5 能复用的开源项目

复用有一条红线：AGPL/GPL 许可证的项目只能学思路、不能抄代码（会有开源传染风险）。表 3-4 按复用深度分档，星标数是 2026 年 9 月 30 日实测。

表 3-4 可复用开源资产

档位	项目（星标/许可证）	怎么用
直接用	reviewdog（9,633 / MIT）	告警回写 PR 评论
直接用	LangGraph（42,507 / MIT）	智能体编排框架
当工具用	Semgrep MCP、CodeQL MCP（官方）	后续做规则生成的工具层
当对照	open-code-review（42,795 / Apache-2.0）	跑基线对比，不引代码
学思路	PR-Agent（13,199 / MIT）	提示词与评论设计参考
学思路	VulnHuntr（约 2,800 / AGPL）	多轮追问的分诊范式，禁止引代码

档位	项目（星标/许可证）	怎么用
学思路	LLMxCPG（USENIX 2025 / GPL）	代码属性图切片思路，禁止引代码
备用	UTBotCpp（186 / Apache-2.0）	C 测试生成后备，已停更，仅评估

3.6 模型怎么选、钱怎么省

原则：分诊量大、单条简单，用便宜的小模型；报告生成量小、要求高，用强模型——这样对冲多智能体约 15 倍的成本放大。低危告警不过模型、只走规则过滤，再设单项目成本上限。起步用 GLM/Qwen/DeepSeek 的 API；将来要内网部署，换 vLLM 在本地跑 Qwen3，架构不用改。国产模型在工具调用能力上已经和闭源模型打平，没有障碍。

3.7 效果怎么评

四项指标：去误报率（判为误报的告警里真误报的占比）、漏判率（真缺陷被当成误报的比例）、分级准确率、单条告警成本。基准用 NIST Juliet 测试集——几万个专门埋好缺陷的现成程序，第一周就下载，挑三到五个缺陷类别，再自己补一些工具查不到的。每次改提示词必须跑回归，防止改好一处、弄坏三处。

3.8 风险与对策

表 3-5 主要风险

风险	后果	对策
大模型胡说（幻觉）	把真缺陷当误报删掉	结论必须附代码证据；高危项人工确认；定期抽检
花销失控	规模化用不起	低危不过模型；小模型分诊；单项目上限
没有评测基准	不知道改好还是改坏	第一周就建 Juliet 评测集；指标脚本化
许可证污染	合规风险	AGPL/GPL 项目禁止引代码
贪多求全	四个维度一起上，全做不精	严格串行：静态缺陷没闭环不开下一个

三条纪律：不急着搭多智能体委员会（分诊是并行单智能体的活，等数据证明需要再加）；AGPL/GPL 代码绝不进仓库；没有评测集就不写提示词。

第 4 章　本周就能开始的事

4.1 三件事，不用写代码

第一件：跑一遍 open-code-review，拿到通用 AI 审查的基线成绩——以后我们的系统要比它更懂嵌入式。第二件：用 cppcheck 扫目标工程，拿一批真实告警，手工试一轮分诊提示词（判断是不是误报、给出证据），顺便估算单条成本。第三件：下载 Juliet 测试集，挑几个缺陷类别，搭好评测集雏形。三件事一天能做完，为第一周开发铺路。

4.2 需要拍板的三个问题

一是部署形态：要不要纯内网部署？这决定用云端 API 还是本地模型。二是规范优先级：先做 MISRA 还是 GJB？决定规则库建设顺序。三是目标工程：用 FreeRTOS、RT-Thread 还是我们自己的工程？建议先用开源工程，好演示也好评测，跑通后再切真实工程。这三个问题都不影响第一周开工。